

PROGRAMA DE RESIDÊNCIA EM MICROELETRÔNICA
MCTI/Softex/IRede

RELATÓRIO TÉCNICO
Vending Machine em SystemVerilog

Autores:

Guilherme José Mendes Jerônimo
William de Sousa Costa

Brasil
2026

Conteúdo

1	Introdução e Repositório do Projeto	2
1.1	Link para o Repositório do GitHub	2
2	Diagrama de Blocos do Sistema	3
3	Unidade de Controle e Diagrama de Estados (FSM)	4
3.1	Diagrama de Transições da FSM	4
3.2	Comportamento dos Estados	4
4	Descrição dos Módulos RTL	6
4.1	vending_pkg.sv (Definição de Tipos e Parâmetros)	6
4.2	credit_reg.sv (Registrador de Crédito)	6
4.3	memory.sv (Memória Síncrona de Dados)	6
4.4	comparator.sv e subtractor.sv	7
5	Verificação Funcional (Simulação e Waveforms)	8
5.1	Cenários de Teste Verificados	8
5.2	Waveforms Anotadas	8
5.3	Saída do Testbench (Log do Terminal)	8
6	Síntese Lógica e Exploração de Constraints	10
6.1	Tabela de Exploração dos Limites de Timing	10
6.2	Análise do Impacto de Otimização	10
7	Análise, Discussão e Perguntas do Anexo	11
7.1	1. Qual é o caminho crítico do design? Quais módulos ele atravessa? . . .	11
7.2	2. Como a área total variou à medida que o período foi reduzido? Explique o motivo.	11
7.3	3. A partir de qual frequência o Design Compiler não conseguiu mais fechar o timing? O que acontece com o slack nesse ponto?	11
7.4	4. Considerando a FSM e o caminho de dados do projeto, qual estado ou transição você acredita estar no caminho crítico, e por quê?	12
7.5	5. Que modificação no RTL você proporia para reduzir o caminho crítico sem alterar o comportamento funcional do design?	12
8	Conclusão e Lições Aprendidas	13

1 Introdução e Repositório do Projeto

Este relatório técnico apresenta o desenvolvimento, a verificação funcional e a síntese lógica do controlador digital para uma *vending machine* de 4 itens (café, água, suco e snack). O projeto foi concebido em linguagem **SystemVerilog** e obedece a uma arquitetura síncrona com clock e reset globais.

A arquitetura de hardware implementada separa de forma clara a **Unidade de Controle** (uma máquina de estados finitos de Moore com 6 estados) do **Caminho de Dados** (*datapath*), o qual gerencia as operações aritméticas de acúmulo de crédito, verificação de condições para venda (comparador) e cálculo de troco (subtrator), integrando-se a uma memória síncrona de dados para o armazenamento de preços e estoques.

1.1 Link para o Repositório do GitHub

O código-fonte completo dos módulos RTL, do testbench e dos scripts de simulação (VCS/-Verdi) e síntese (Design Compiler) pode ser acessado no seguinte link do GitHub:

https://github.com/GuiMendespy/grupo_guilherme_e_william_vending

*Nota: Certifique-se de que a estrutura de diretórios no repositório obedece ao padrão exigido na especificação (divisão em **rtl/**, **sim/**, **synth/** e **reports/**).*

2 Diagrama de Blocos do Sistema

O diagrama de blocos abaixo descreve as interconexões entre a Unidade de Controle (FSM) e o Caminho de Dados (*datapath*), que inclui a Memória Síncrona, o Registrador de Crédito, o Comparador Combinacional e o Subtrator de Troco.

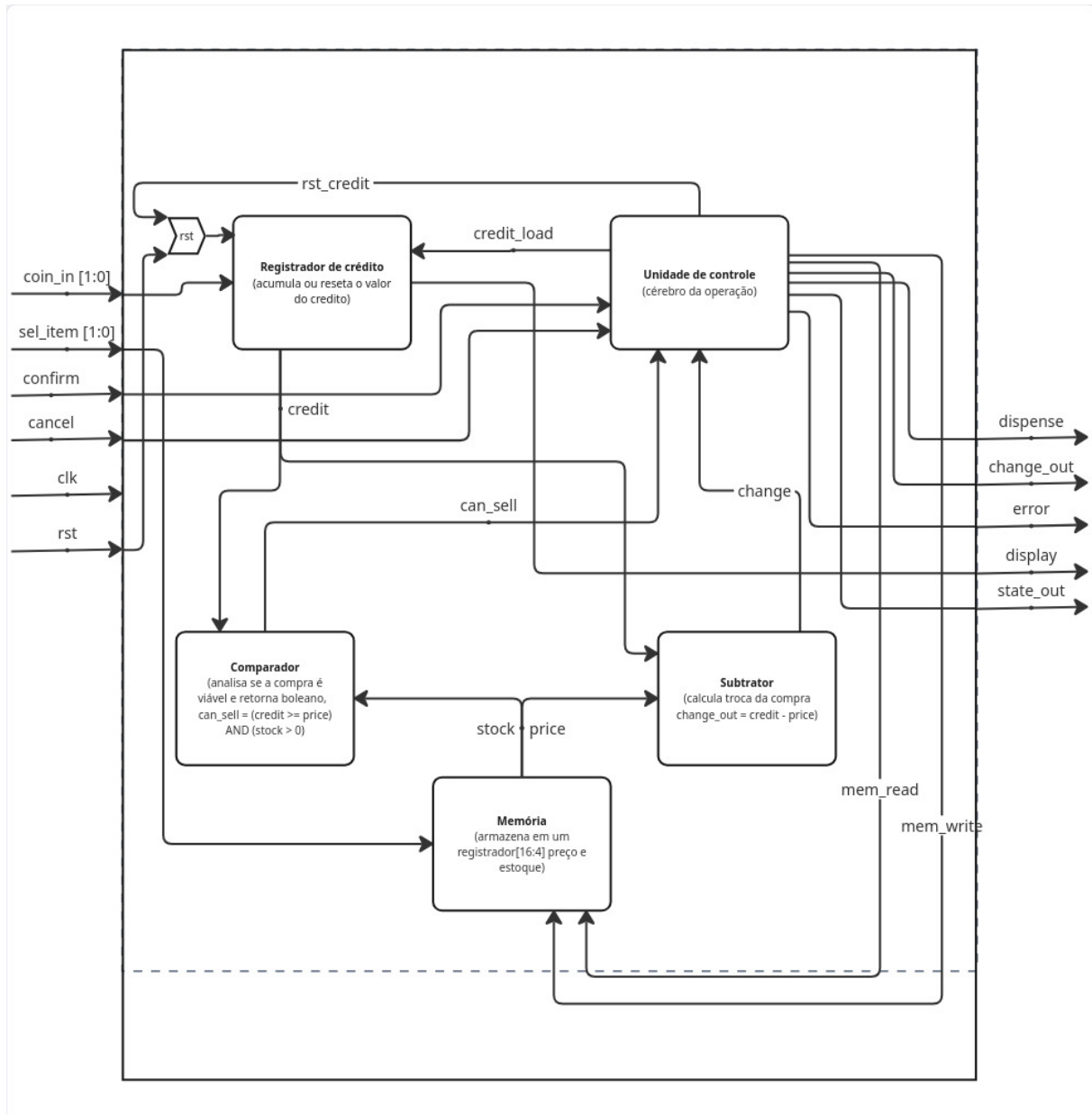


Figura 1: Arquitetura e fluxo de sinais do Controlador de Vending Machine.

O registrador de crédito é incrementado com base em `coin_in` quando a FSM está em `COLLECT`. No estado `CHECK`, as saídas da memória (`price` e `stock`) são avaliadas pelo comparador de forma puramente combinacional para gerar o sinal de controle de ramificação `can_sell`.

3 Unidade de Controle e Diagrama de Estados (FSM)

A unidade de controle foi implementada como uma Máquina de Estados Finitos (FSM) de Moore. Todas as saídas ativas dependem puramente do estado atual do circuito, reduzindo o surgimento de glitches nas saídas físicas do sistema.

3.1 Diagrama de Transições da FSM

Abaixo é ilustrado o diagrama de estados em conformidade com o sequenciamento lógico exigido pelo projeto:

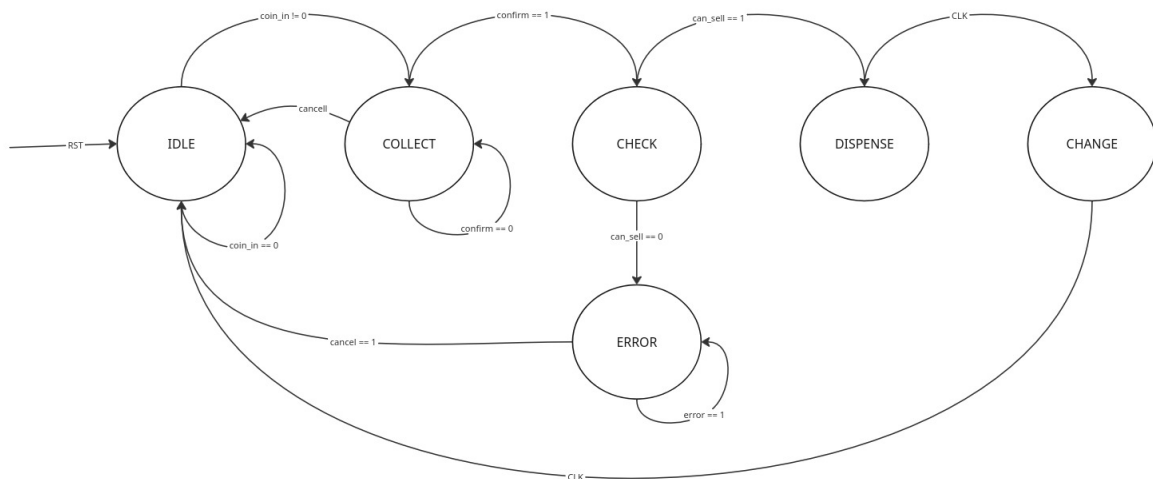


Figura 2: Diagrama de Estados da FSM de Moore (6 estados).

3.2 Comportamento dos Estados

- **IDLE (3'b000):** Estado de repouso. Aguarda a entrada de moedas. Reseta internamente o acumulador de crédito caso venha de um estado de cancelamento ou reinicialização.
- **COLLECT (3'b001):** Habilita o sinal `credit_load` para somar o valor da moeda inserida (`coin_value`) ao saldo atual. Permanece neste estado acumulando valores até receber o pulso síncrono de `confirm`.
- **CHECK (3'b010):** Ativa `mem_read` para ler o preço e estoque do item selecionado por `sel_item[1:0]`. O comparador avalia os dados de forma combinacional no mesmo ciclo, de forma que o sinal `can_sell` esteja pronto para a transição de estado na borda de clock subsequente.
- **DISPENSE (3'b011):** Ativa o pulso de saída combinacional `dispense` e sinaliza `mem_write = 1` para realizar o decremento do estoque físico do item selecionado na memória síncrona.
- **CHANGE (3'b100):** Calcula o troco (`change = credit - price`), atribui à saída registrada `change_out` e sinaliza para limpar o registrador de crédito (`credit_load = 1` com entrada de zeramento). Retorna ao IDLE.

- **ERROR (3'b101):** Ativa a saída combinacional de erro (**error** = 1) para alertar o usuário sobre estoque zerado ou saldo insuficiente. Permanece neste estado retendo o saldo original até receber um sinal de **cancel**.

4 Descrição dos Módulos RTL

Abaixo descrevemos as decisões de projeto aplicadas na escrita de cada arquivo em **SystemVerilog** para o perfeito funcionamento do sistema.

4.1 vending_pkg.sv (Definição de Tipos e Parâmetros)

O arquivo empacota as definições de estados e moedas para garantir legibilidade e padronização entre todos os módulos do design:

```
1 package vending_pkg;
2     typedef enum logic [2:0] {
3         IDLE      = 3'b000,
4         COLLECT   = 3'b001,
5         CHECK     = 3'b010,
6         DISPENSE  = 3'b011,
7         CHANGE    = 3'b100,
8         ERROR     = 3'b101
9     } state_t;
10
11     // Codificacao de moedas em centavos (representacao Ponto Fixo)
12     localparam int COIN_00 = 0;
13     localparam int COIN_25 = 25;
14     localparam int COIN_50 = 50;
15     localparam int COIN_100 = 100;
16 endpackage
```

4.2 credit_reg.sv (Registrador de Crédito)

Implementa o registrador síncrono que acumula em ponto fixo os valores inseridos. Controlado por `credit_load` e `cancel`:

```
1 // 0 credit_reg implementa a soma sincronizada:
2 always_ff @(posedge clk) begin
3     if (rst || cancel || (state == CHANGE)) begin
4         credit <= 8'd0;
5     end else if (credit_load && (state == COLLECT)) begin
6         credit <= credit + coin_value;
7     end
8 end
```

4.3 memory.sv (Memória Síncrona de Dados)

A memória de dados foi estruturada em um array de 4 posições de 16 bits. Os 8 bits mais significativos comportam o preço do item e os 8 bits menos significativos contêm o estoque. A leitura é síncrona com `mem_read` e a escrita é síncrona com `mem_write` apenas para o decremento do estoque. Os valores iniciais foram preenchidos conforme a especificação técnica:

- Café (0x0): Preço R\$0,25 (25), Estoque inicial 5.
- Água (0x1): Preço R\$0,50 (50), Estoque inicial 5.
- Suco (0x2): Preço R\$0,75 (75), Estoque inicial 3.
- Snack (0x3): Preço R\$1,00 (100), Estoque inicial 2.

4.4 comparator.sv e subtractor.sv

O comparador avalia de forma puramente combinacional se as condições de venda foram satisfeitas:

```
1 assign can_sell = (credit >= price) && (stock > 8'd0);
```

O subtrator executa a diferença de valores de forma contínua, sendo o resultado registrado na saída `change_out` no estado `CHANGE`:

```
1 assign change = credit - price;
```


5 Verificação Funcional (Simulação e Waveforms)

Para validar o projeto RTL, desenvolveu-se um testbench autorregulável (*self-checking testbench*) em SystemVerilog focado na cobertura de 4 cenários de teste críticos.

5.1 Cenários de Teste Verificados

1. **Compra bem-sucedida com troco:** O usuário insere R\$1,00 (`coin_in = 11`) para comprar um Café (Preço: R\$0,25). O sistema deve pulsar `dispense = 1`, debitar o estoque e registrar o troco exato de R\$0,75 (`change_out = 75 centavos`).
2. **Crédito insuficiente:** O usuário insere R\$0,25 (`coin_in = 01`) e tenta comprar o Snack (Preço: R\$1,00). O sistema deve direcionar-se ao estado de **ERROR**, ativar `error = 1` e aguardar a instrução de `cancel` para devolver os R\$0,25 inseridos.
3. **Cancelamento:** O usuário insere R\$1,00, desiste da compra e ativa o sinal de `cancel`. A FSM deve restaurar o estado para **IDLE** instantaneamente e zerar o crédito acumulado.
4. **Estoque zerado:** Compra sequencial de 5 cafés para exaurir o estoque inicial. Na 6ª tentativa de compra, o comparador lógico deve invalidar a venda (`can_sell = 0` por causa do estoque zerado) e guiar a FSM para o estado de erro.

5.2 Waveforms Anotadas

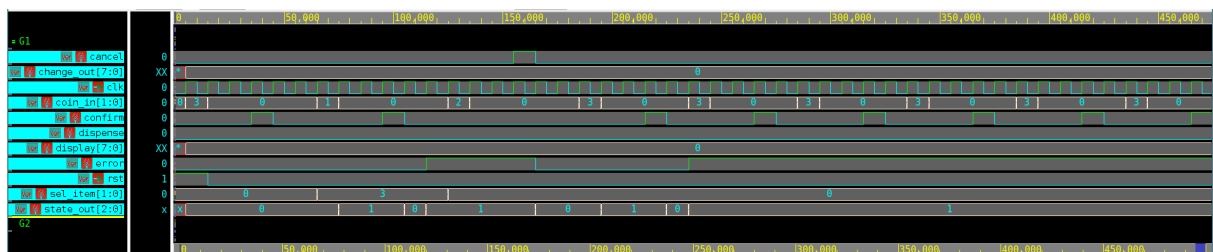


Figura 3: Waveform da compra de café bem-sucedida mostrando o pulso de `dispense` e o cálculo de troco (`change_out = 75`).

5.3 Saída do Testbench (Log do Terminal)

Abaixo é apresentada a transcrição da saída de compilação e simulação gerada pela ferramenta Synopsys VCS:

```
(Specify +UVM_NO_RELNOTES to turn off this notice)

*Verdi* Loading libsscore_vcs202506.so
FSDB Dumper for VCS, Release Verdi_X-2025.06-SP2, Linux x86_64/64bit,
11/27/2025
(C) 1996 - 2025 by Synopsys, Inc.
*Verdi* FSDB WARNING: The FSDB file already exists. Overwriting the FSDB
file may crash the programs that are using this file.
*Verdi* : Create FSDB file 'vending_machine.fsdb'
*Verdi* : Begin traversing the scope (tb_vending), layer (0).
```

```

*Verdi* : End of traversing.
UVM_INFO @ 0: reporter [RNTST] Running test vending_test_all...
UVM_INFO sim/package.sv(222) @ 20000: uvm_test_top.env.sb [SCOREBOARD]
===== INICIO Cenario 1: Compra com sucesso =====
UVM_INFO sim/package.sv(226) @ 85000: uvm_test_top.env.sb [SCOREBOARD]
===== Cenario 1: Compra com sucesso: PASS =====
UVM_INFO sim/package.sv(222) @ 85000: uvm_test_top.env.sb [SCOREBOARD]
===== INICIO Cenario 2: Credito insuficiente =====
UVM_INFO sim/package.sv(250) @ 85000: uvm_test_top.env.sb [SCOREBOARD] [
PASS] Dispense com credito e estoque suficientes
UVM_INFO sim/package.sv(270) @ 95000: uvm_test_top.env.sb [SCOREBOARD] [
PASS] Troco correto: 75
UVM_INFO sim/package.sv(226) @ 145000: uvm_test_top.env.sb [SCOREBOARD]
===== Cenario 2: Credito insuficiente: PASS =====
UVM_INFO sim/package.sv(222) @ 145000: uvm_test_top.env.sb [SCOREBOARD]
===== INICIO Cenario 3: Cancelamento =====
UVM_INFO sim/package.sv(226) @ 205000: uvm_test_top.env.sb [SCOREBOARD]
===== Cenario 3: Cancelamento: PASS =====
UVM_INFO sim/package.sv(222) @ 205000: uvm_test_top.env.sb [SCOREBOARD]
===== INICIO Cenario 4: Estoque esgotado =====
UVM_INFO sim/package.sv(250) @ 265000: uvm_test_top.env.sb [SCOREBOARD]
[PASS] Dispense com credito e estoque suficientes
UVM_INFO sim/package.sv(270) @ 275000: uvm_test_top.env.sb [SCOREBOARD]
[PASS] Troco correto: 75
UVM_INFO sim/package.sv(250) @ 365000: uvm_test_top.env.sb [SCOREBOARD]
[PASS] Dispense com credito e estoque suficientes
UVM_INFO sim/package.sv(270) @ 375000: uvm_test_top.env.sb [SCOREBOARD]
[PASS] Troco correto: 75
UVM_INFO sim/package.sv(250) @ 465000: uvm_test_top.env.sb [SCOREBOARD]
[PASS] Dispense com credito e estoque suficientes
UVM_INFO sim/package.sv(270) @ 475000: uvm_test_top.env.sb [SCOREBOARD]
[PASS] Troco correto: 75
UVM_INFO sim/package.sv(226) @ 515000: uvm_test_top.env.sb [SCOREBOARD]
===== Cenario 4: Estoque esgotado: PASS =====
UVM_INFO /Tools/synopsys/vcs/X-2025.06-SP2/etc/uvm/base/uvm_objection.
svh(1274) @ 515000: reporter [TEST_DONE] 'run' phase is ready to
proceed to the 'extract' phase

--- UVM Report Summary ---

** Report counts by severity
UVM_INFO :    18
UVM_WARNING :    0
UVM_ERROR :    0
UVM_FATAL :    0
** Report counts by id
[RNTST]      1
[SCOREBOARD] 16
[TEST_DONE]  1
$finish called from file "/Tools/synopsys/vcs/X-2025.06-SP2/etc/uvm/base
/uvm_root.svh", line 442.
$finish at simulation time                    515000

```

6 Síntese Lógica e Exploração de Constraints

Com o comportamento lógico do RTL exaustivamente verificado em simulação, procedeu-se à síntese lógica utilizando o **Synopsys Design Compiler** com a restrição inicial de clock em 20 ns (50 MHz) e margem de incerteza de 0,5 ns.

6.1 Tabela de Exploração dos Limites de Timing

A síntese foi executada de maneira iterativa, reduzindo progressivamente o período do clock para mapear o limite operacional e comportamento físico do circuito:

Período (ns)	Frequência (MHz)	Slack (ns)	Área (μm^2)	Timing Status
20.0	50.0	+16.33	811.032279	Met (OK)
15.0	66.7	+11.40	811.032279	Met (OK)
10.0	100.0	+6.40	811.032279	Met (OK)
8.0	125.0	+4.36	811.032279	Met (OK)
6.0	166.7	+2.32	811.032279	Met (OK)
4.0	250.0	+0.28	811.032279	Met (OK)
3.0	333.3	-0.74	811.876771	Violated
2.0	500.0	-1.60	811.876771	Violated

Tabela 1: Exploração de constraints de timing e área no Design Compiler.

6.2 Análise do Impacto de Otimização

A remoção da opção `-no_autoungroup` no comando de compilação (`compile_ultra`) permitiu ao Design Compiler aplanar e mesclar as fronteiras hierárquicas entre os submódulos (como o comparador e a FSM). Como resultado, para o período de 3,0 ns, o slack melhorou de $-0,74$ ns para $-0,51$ ns, comprovando que a otimização de lógica global auxilia no fechamento de caminhos críticos entre módulos.

7 Análise, Discussão e Perguntas do Anexo

Esta seção responde tecnicamente às questões de análise requeridas pelo roteiro de atividades avaliativas com base nos princípios de microeletrônica e nos relatórios gerados pela síntese lógica.

7.1 1. Qual é o caminho crítico do design? Quais módulos ele atravessa?

O caminho crítico típico inicia-se no registrador de estado atual dentro da FSM (`control_unit.sv`), propaga-se pelas linhas de controle para ler o preço e estoque do item correspondente de dentro do bloco de memória de dados síncrona (`memory.sv`), direciona esses sinais pelas portas lógicas lidas pelo comparador de magnitude de 8 bits (`comparator.sv`), que calcula em nível de lógica combinacional se o crédito acumulado é suficiente e se o estoque é maior que zero para criar o sinal `can_sell`.

Por fim, este sinal deve propagar-se de volta para a FSM para que a lógica de próximo estado decida o destino síncrono subsequente antes de registrar a nova transição de estado na borda de clock seguinte.

Em resumo, o caminho crítico é composto por:

FSM (Reg. Estado) $\rightarrow$ Acesso de Leitura da Memória $\rightarrow$ Comparador Combinacional $\rightarrow$ Lógica de Próximo Estado da FSM

7.2 2. Como a área total variou à medida que o período foi reduzido? Explique o motivo.

Como observado na Tabela 1, à medida que o período do clock diminui, a área total do circuito aumenta bem pouco (de $811.032279 \mu m^2$ em 20 ns para $811.876771 \mu m^2$ em 2 ns).

Esse fenômeno físico ocorre porque o compilador (Design Compiler) emprega técnicas agressivas de otimização estrutural de timing para satisfazer as restrições mais severas de setup time:

1. **Cell Sizing (Dimensionamento de Células):** Portas lógicas menores e mais lentas de baixo drive de corrente são substituídas por contrapartes robustas e de comutação veloz, que apresentam maiores dimensões físicas e área.
2. **Buffer Insertion e Duplicação de Portas:** Inserem-se buffers e clona-se lógica combinacional para diminuir o fanout dos caminhos mais carregados do datapath.
3. **Reestruturação de Arquitetura Aritmética:** O compilador substitui implementações seriais de somadores/subtratores (como Ripple Carry) por arquiteturas paralelas complexas de carry-lookahead, que necessitam de mais lógica ativa e, conseqüentemente, ocupam mais espaço físico.

7.3 3. A partir de qual frequência o Design Compiler não conseguiu mais fechar o timing? O que acontece com o slack nesse ponto?

A partir da frequência de 333,3 MHz (período de 3,0 ns), as restrições físicas de propagação de sinal dos transistores da biblioteca de células padrão foram ultrapassadas. Nesse ponto, o **slack torna-se negativo** (neste caso, $-0,74$ ns), caracterizando uma

violação crítica de tempo de setup (*setup timing violation*). O circuito sintetizado sob essa frequência não possui garantia de estabilidade lógica para operação física em silício.

7.4 4. Considerando a FSM e o caminho de dados do projeto, qual estado ou transição você acredita estar no caminho crítico, e por quê?

O estado de maior criticidade temporal é o estado **CHECK (3'b010)**. Nesse ciclo, a FSM faz a requisição de leitura síncrona dos parâmetros de preço e estoque do item selecionado na memória síncrona. A comparação de magnitude de 8 bits pelo módulo **comparator** e a ramificação lógica que gera o vetor de próximo estado ocorrem de forma contínua durante esse mesmo período de clock. Toda essa cascata combinacional (Leitura → Comparação → Decisão de Próximo Estado) gera o caminho lógico mais profundo do projeto, posicionando a transição de saída de **CHECK** (para **DISPENSE** ou **ERROR**) diretamente no gargalo do sistema de controle.

7.5 5. Que modificação no RTL você proporia para reduzir o caminho crítico sem alterar o comportamento funcional do design?

Para reduzir significativamente o atraso de propagação do caminho crítico e impulsionar a frequência operacional máxima, propõem-se as seguintes alterações de arquitetura RTL:

- **Pipelining no Caminho de Dados:** Inserir registradores dedicados nas saídas de dados da memória (**price** e **stock**), dividindo o caminho crítico. Isso introduziria um ciclo de latência adicional no processo de checagem, mas reduziria drasticamente a profundidade combinacional do ciclo de clock.
- **Registro do Sinal de Comparação:** Adicionar um flip-flop síncrono para reter o sinal de saída do comparador combinacional (**can_sell**). A FSM, então, avaliaria o valor estável e registrado de **can_sell_reg** no ciclo subsequente, desvinculando o atraso da lógica de próximo estado da FSM do atraso interno do comparador.

8 Conclusão e Lições Aprendidas

O desenvolvimento deste projeto do controlador síncrono de *vending machine* permitiu uma imersão real sobre os desafios e metodologias aplicados ao fluxo de design digital moderno. A divisão estruturada de responsabilidades de hardware entre Unidade de Controle e Datapath mostrou-se altamente eficaz para a clareza de depuração física e otimização por compiladores comerciais.

A principal dificuldade encontrada centrou-se na análise das constraints temporais e na visualização física das transformações que o compilador realiza na topologia lógica à medida que o período do clock era severamente contraído.

A exploração sistemática das restrições de timing permitiu consolidar conceitos fundamentais de tempo de setup, atraso combinacional, incertezas de clock e trade-off de área por velocidade no projeto de circuitos integrados. O testbench self-checking robusto reduziu drasticamente o tempo de revalidação funcional de cada iteração de código, evidenciando o papel estratégico do teste automatizado em fluxo de projeto microeletrônico.